



Tecnológico de Monterrey

Campus Monterrey

Reflexión Evidencia 1

Diego Villanueva Fernandez
A01199495

David Alonso Cantú Delgado
Programación de estructuras de datos y algoritmos fundamentales
Grupo 607

20 de Sep del 2026

Reflexión sobre la Evidencia 1

1. ¿Por qué es importante ordenar y buscar registros de seguridad?

Porque cuando tienes miles de logs desordenados encontrar algo específico se vuelve muy tardado. Al ordenarlos por fecha ya puedes ver qué pasó primero, qué pasó después y además usar búsqueda binaria para encontrar rangos mucho más rápido.

2. ¿Qué diferencia noté entre los algoritmos de ordenamiento?

La diferencia principal fue que no todos reaccionan igual al estado inicial de los datos. Swap y Bubble fueron muy lentos con el archivo desordenado, Merge fue bastante constante, Insertion mejoró muchísimo cuando el archivo ya estaba casi ordenado y Quick Sort hizo lo contrario por la forma en la que elegía el pivote.

3. ¿Qué comportamiento cambió más entre los dos archivos?

Insertion Sort pasó de 248.895 ms a 2.37004 ms, entonces ahí sí se notó muchísimo que tener los datos casi ordenados le ayuda. Quick Sort pasó de 51.8463 ms a 3053.51 ms porque usaba el último elemento como pivote y eso generó particiones muy desbalanceadas.

4. ¿Qué aprendí de la complejidad Big-O?

Que Big-O sirve para entender cómo crece un algoritmo, pero no significa que dos algoritmos con la misma complejidad van a tardar exactamente lo mismo. Selection Sort también es $O(n^2)$, pero en mis pruebas fue muchísimo más rápido que Swap y Bubble porque hace menos movimientos de registros.

Resultados y decisiones

5. ¿Mis predicciones coincidieron con los resultados?

En la mayoría sí. Merge, Shell, Insertion y Quick Sort se comportaron como esperaba en los casos más importantes. Donde me equivoqué fue en algunos algoritmos $O(n^2)$, porque pensé que necesariamente iban a ser lentos solo por su complejidad y los tiempos reales mostraron que también importan mucho las constantes y cuántos movimientos hacen.

6. ¿Qué algoritmo usaría dependiendo del caso?

Si los datos ya están casi ordenados, Insertion Sort funcionó muy bien. Si no sé cómo vienen los datos, Merge Sort me parece más estable porque se mantuvo alrededor de 40 ms en ambos archivos. También Shell Sort tuvo tiempos muy buenos en los dos casos. En esta implementación evitaría Quick Sort si los datos ya están casi ordenados y se sigue usando el último elemento como pivote.

7. ¿Qué aprendí con la búsqueda por rango?

Que no basta con encontrar una fecha exacta. Había que definir bien qué pasa si los límites no existen y qué hacer con timestamps repetidos. Por eso usé un lower bound para encontrar el primer valor mayor o igual al inicio y un upper bound para llegar hasta el primer valor mayor al final. Así el rango queda inclusivo y no se pierde ningún duplicado.

8. ¿Qué pasó con los timestamps duplicados?

Probé Oct 02 2024 23:04:24 como inicio y fin y el programa regresó dos registros. Eso confirmó que la búsqueda sí incluye todas las apariciones con el mismo timestamp y no solo la primera que encuentra.

Uso de inteligencia artificial

9. ¿Para qué usé IA durante el desarrollo?

La usé principalmente para entender mejor cómo estructurar el programa, revisar complejidades y pensar cómo hacer la búsqueda por rango sin perder duplicados. No copié una solución completa y ya; fui agregando partes poco a poco, probándolas y cambiando lo que no funcionaba como yo necesitaba.

10. Prompts importantes que utilicé

- Cómo convertir la fecha y hora de cada log a un valor que pudiera comparar fácilmente para ordenar los registros.
- Cómo hacer una búsqueda binaria por rango que incluyera todos los timestamps duplicados en los límites.
- Cómo comparar los siete algoritmos y qué complejidad tenía cada implementación para poder hacer una predicción antes de correrla.

11. ¿Qué modifiqué de lo que me sugirió la IA?

Adapté las rutas para que funcionaran desde CLion, agregué validación de entradas, hice que se pudieran correr varias pruebas sin reiniciar el programa, conservé el formato original de las fechas y ajusté la búsqueda para usar límites inclusivos. También fui cambiando las predicciones con base en cómo estaba implementado cada algoritmo y no solo por la teoría general.

12. Error o corrección importante de IA

Una corrección concreta fue con Bubble Sort. Al principio se tomó como referencia el mejor caso típico $O(n)$, pero revisando mi código vi que mi implementación no tenía una bandera de swapped ni un early stop. Eso significa que aunque el archivo ya esté ordenado, sigue recorriendo los ciclos completos, así que en mi implementación el mejor caso también es $O(n^2)$. Lo corregí antes de dejarlo en el programa y en el análisis.

Conclusión personal

Esta evidencia me ayudó a ver que la teoría sí sirve, pero que correr y medir los algoritmos cambia mucho la forma de entenderlos. Lo que más me quedó fue que el mismo algoritmo puede comportarse totalmente diferente dependiendo de cómo vienen los datos y de pequeños detalles de implementación.